

Федеральное государственное автономное образовательное
учреждение
высшего образования «Санкт-Петербургский национальный
исследовательский Университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №2
По вычислительной математике
Вариант №2

Выполнил: Гладышев Иван Станиславович
Группа: Р3213

Принимающий работу: Бострикова Дарья Константиновна

Санкт-Петербург 2026

Содержание

1	Цель работы	3
2	Порядок выполнения работы	4
2.1	Решение нелинейного уравнения	4
2.2	Решение системы нелинейных уравнений	4
2.3	Программная реализация	4
3	Рабочие формулы используемых методов	5
3.1	Метод хорд	5
3.2	Метод секущих	6
3.3	Метод простой итерации	7
3.4	Метод половинного деления	9
3.5	Метод Ньютона для систем нелинейных уравнений	10
3.6	Метод простой итерации для систем нелинейных уравнений . .	12
4	Графики функций на исследуемом интервале	14
4.1	График нелинейного уравнения	14
4.2	График системы нелинейных уравнений	15
5	Вычислительная часть 1. Решение нелинейного уравнения	16
5.1	Задание	16
5.2	Интервалы изоляции корней	16
5.3	Уточнение левого корня (метод хорд)	17
5.4	Уточнение центрального корня (метод секущих)	17
5.5	Уточнение правого корня (метод простой итерации)	17
6	Вычислительная часть 2. Решение системы нелинейных уравнений через метод Ньютона	19
6.1	Система	19
6.2	Приведение к стандартному виду	19
6.3	Матрица Якоби	19
6.4	Система линейных уравнений для нахождения приращений . .	19
6.5	Итерационный процесс	20
7	Листинг программы	22
7.1	Класс NonlinearSolve	22
7.2	Класс SystemSolver	24
8	Примеры и результаты выводов программы	25
8.1	Тест 1: одно уравнение с несколькими корнями на отрезке . . .	25
8.2	Тест 2: система, где файл ввода отсутствует	26

8.3	Тест 3: некорректные значения ввода	26
8.4	Тест 4: В методе Ньютона производная почти равна 0	28
8.5	Тест 5: в методе простой итерации производная меняет знак (не сходится)	29
9	Выводы	31

1 Цель работы

Изучить численные методы решения нелинейных уравнений и их систем, найти корни заданного нелинейного уравнения и системы нелинейных уравнений, выполнить программную реализацию методов.

2 Порядок выполнения работы

2.1 Решение нелинейного уравнения

1. Отделить корни нелинейного уравнения $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$ графически
2. Определить интервалы, в которых лежат три корня уравнения
3. Уточнить корни с точностью $\varepsilon = 10^{-2}$, используя:
 - Метод хорд для левого корня
 - Метод секущих для центрального корня
 - Метод простой итерации для правого корня
4. Результаты вычислений занести в таблицы итерационного процесса
5. Для метода простой итерации проверить условие сходимости на выбранном интервале

2.2 Решение системы нелинейных уравнений

1. Отделить корни системы графически:

$$\begin{cases} \tan(xy + 0.1) = x^2 \\ x^2 + 2y^2 = 1 \end{cases}$$

2. Решить систему методом Ньютона с точностью $\varepsilon = 0.01$
3. Результаты итерационного процесса занести в таблицу

2.3 Программная реализация

1. Реализовать следующие численные методы:
 - (a) Метод половинного деления для нелинейного уравнения
 - (b) Метод Ньютона для нелинейного уравнения
 - (c) Метод простой итерации для нелинейного уравнения
 - (d) Метод простой итерации для систем нелинейных уравнений
2. Предусмотреть ввод исходных данных с клавиатуры или из файла
3. Реализовать вывод результатов на экран или в файл

3 Рабочие формулы используемых методов

3.1 Метод хорд

Рабочая формула метода:

$$x_i = a_i - \frac{b_i - a_i}{f(b_i) - f(a_i)} \cdot f(a_i) \quad (1)$$

или эквивалентная форма:

$$x_i = \frac{a_i f(b_i) - b_i f(a_i)}{f(b_i) - f(a_i)} \quad (2)$$

Блок-схема алгоритма метода хорд:

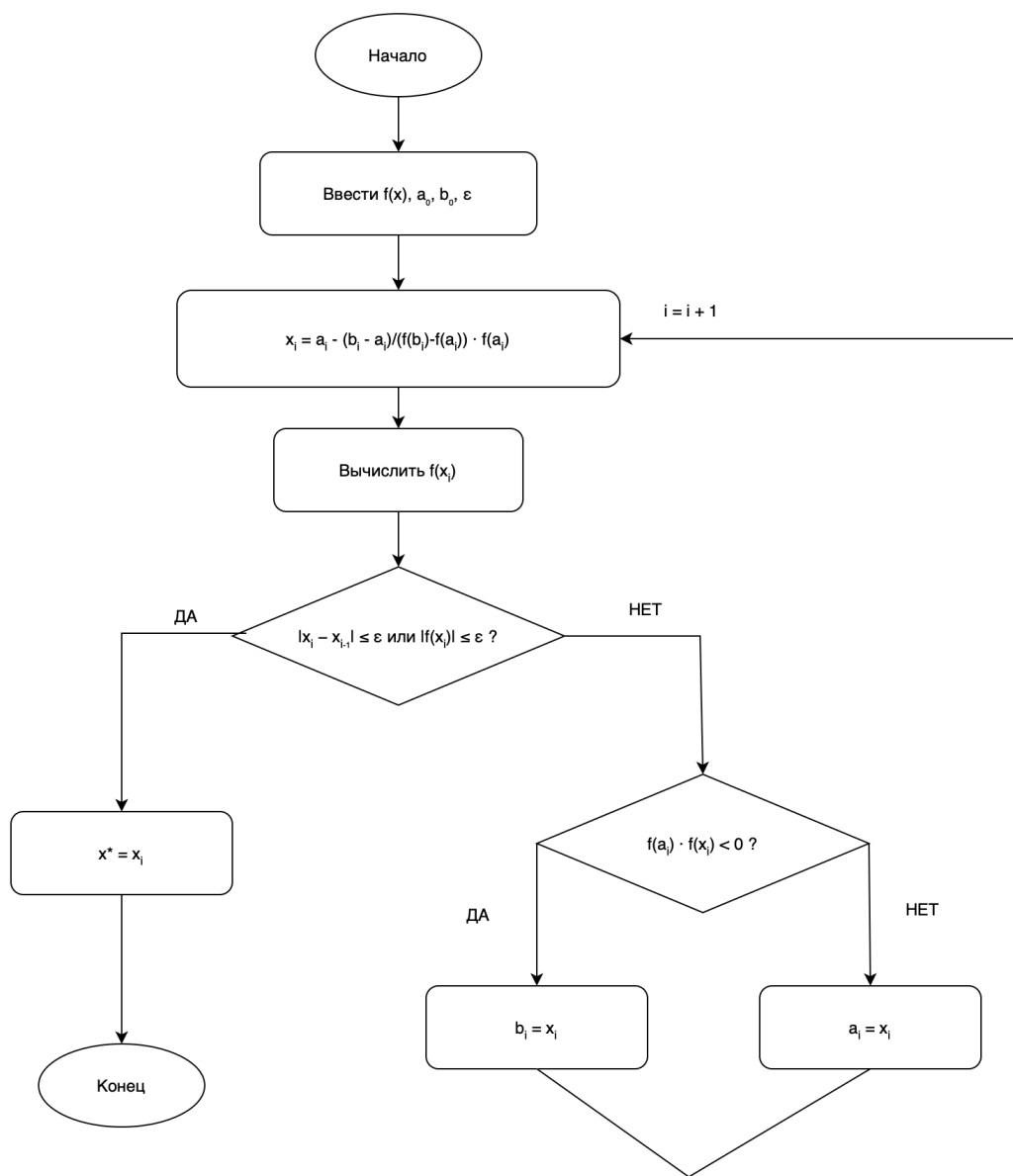


Рис. 1: Блок-схема метода хорд

3.2 Метод секущих

Рабочая формула метода:

$$x_{i+1} = x_i - \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})} \cdot f(x_i), \quad i = 1, 2, \dots \quad (3)$$

Критерий окончания итерационного процесса:

$$|x_n - x_{n-1}| \leq \varepsilon \quad \text{или} \quad |f(x_n)| \leq \varepsilon \quad (4)$$

Приближенное значение корня: $x^* = x_n$

Блок-схема алгоритма метода секущих:

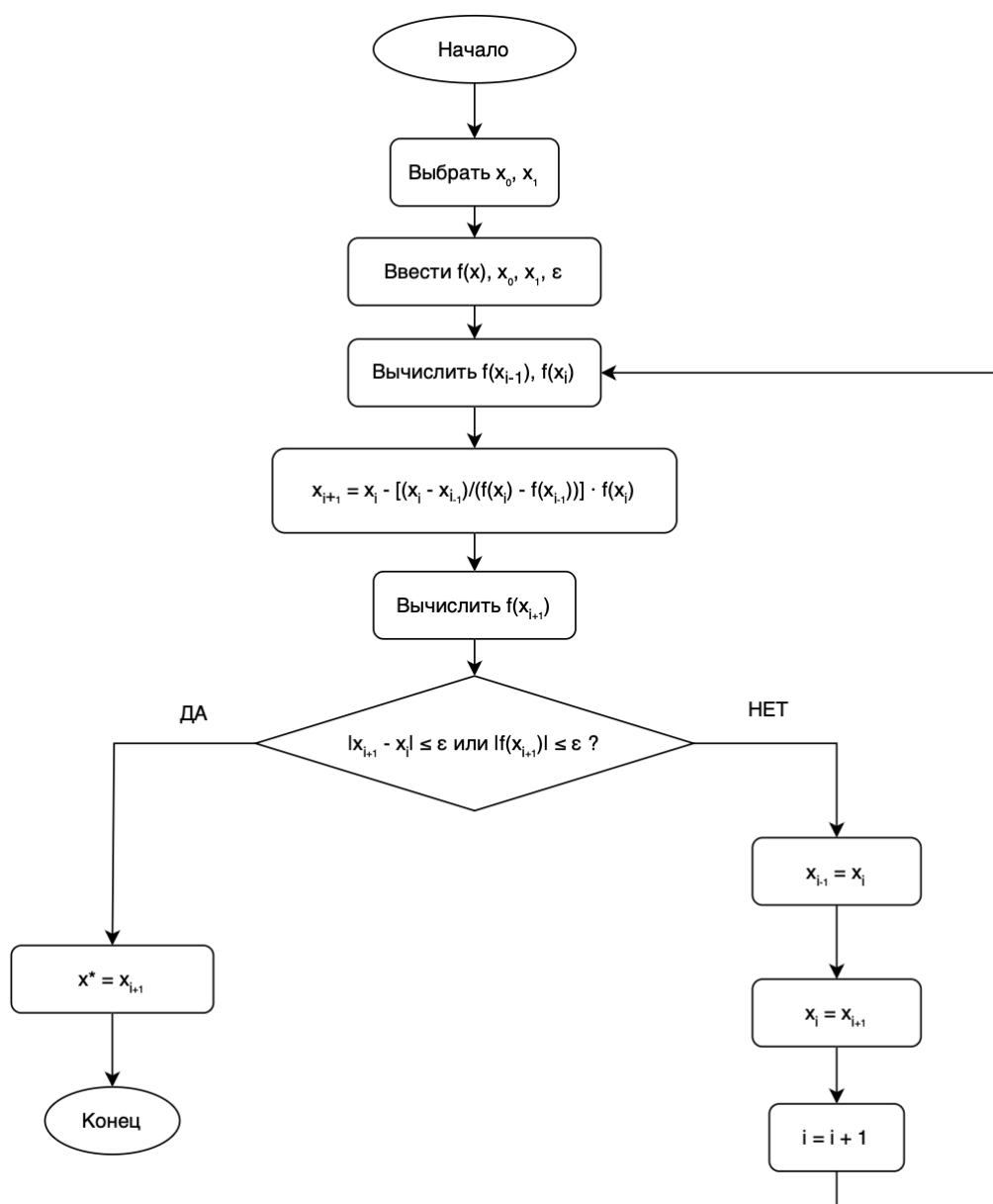


Рис. 2: Блок-схема метода секущих

3.3 Метод простой итерации

Идея метода: уравнение $f(x) = 0$ приводится к эквивалентному виду $x = \varphi(x)$, где $\varphi(x)$ — итерационная функция.

Рабочая формула метода:

$$x_{i+1} = \varphi(x_i) \quad (5)$$

Блок-схема алгоритма метода простой итерации:

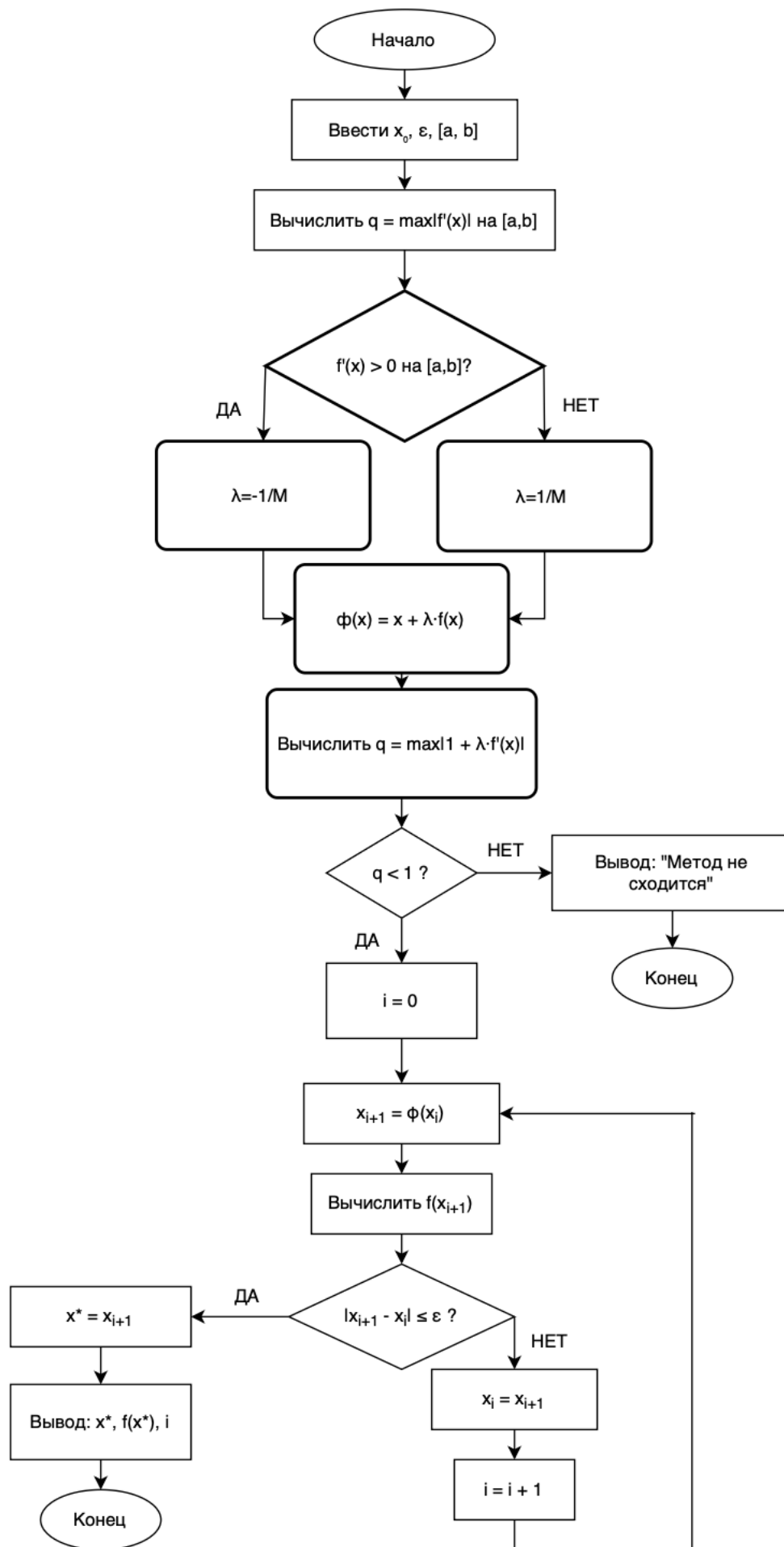


Рис. 3: Блок-схема метода простой итерации

3.4 Метод половинного деления

Идея метода: начальный интервал изоляции корня делится пополам, и в качестве нового интервала выбирается та половина, на концах которой функция имеет разные знаки.

Рабочая формула метода:

$$x_i = \frac{a_i + b_i}{2} \quad (6)$$

Критерий окончания итерационного процесса:

$$|a_n - b_n| \leq \varepsilon \quad \text{или} \quad |f(x_n)| \leq \varepsilon \quad (7)$$

Приближенное значение корня:

$$x^* = \frac{a_n + b_n}{2} \quad \text{или} \quad x^* = a_n \quad \text{или} \quad x^* = b_n \quad (8)$$

Блок-схема алгоритма метода половинного деления:

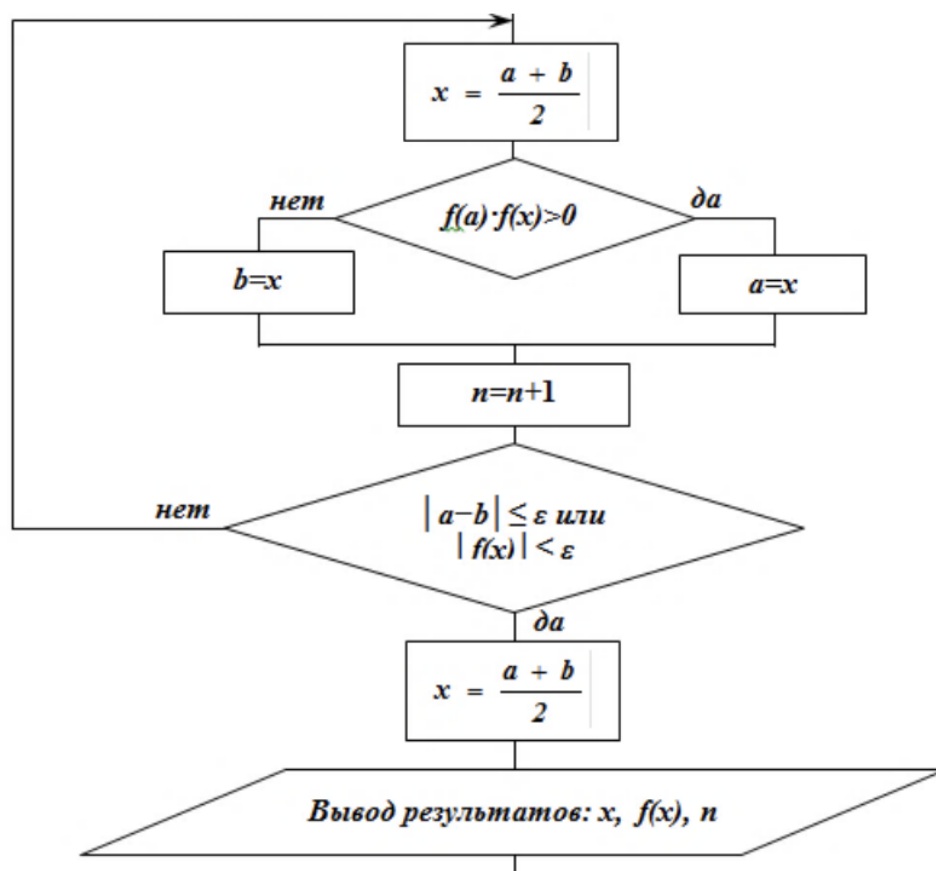


Рис. 4: Блок-схема метода половинного деления

3.5 Метод Ньютона для систем нелинейных уравнений

Постановка задачи: Дана система нелинейных уравнений:

[illegible]

Идея метода: Линеаризация системы с помощью разложения в ряд Тейлора и решения системы линейных уравнений относительно приращений.

Матрица Якоби:

$$J(x) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1} & \frac{\partial F_1}{\partial x_2} & \dots & \frac{\partial F_1}{\partial x_n} \\ \frac{\partial F_2}{\partial x_1} & \frac{\partial F_2}{\partial x_2} & \dots & \frac{\partial F_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial F_n}{\partial x_1} & \frac{\partial F_n}{\partial x_2} & \dots & \frac{\partial F_n}{\partial x_n} \end{pmatrix} \quad (10)$$

Рабочая формула метода:

$$J(\vec{x}^{(k)}) \cdot \Delta \vec{x}^{(k)} = -\vec{F}(\vec{x}^{(k)}) \quad (11)$$

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + \Delta \vec{x}^{(k)} \quad (12)$$

Для системы второго порядка:

$$\begin{cases} f(x, y) = 0 \\ g(x, y) = 0 \end{cases} \quad (13)$$

Матрица Якоби:

$$J(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{pmatrix} \quad (14)$$

Система линейных уравнений для нахождения приращений:

$$\begin{pmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} -f(x, y) \\ -g(x, y) \end{pmatrix} \quad (15)$$

Итерационные формулы:

$$x_{k+1} = x_k + \Delta x_k, \quad y_{k+1} = y_k + \Delta y_k \quad (16)$$

Блок-схема алгоритма метода Ньютона для систем:

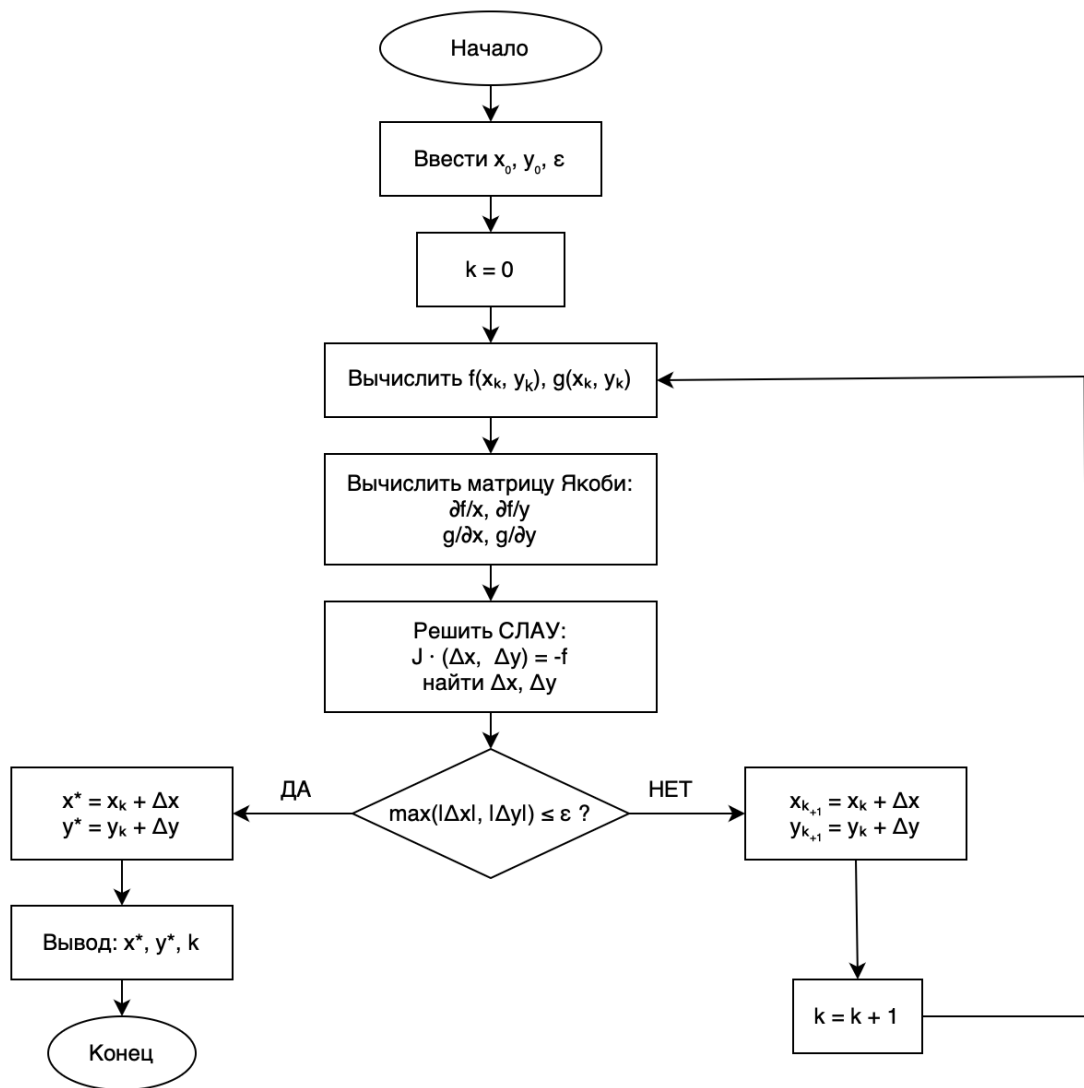


Рис. 5: Блок-схема метода Ньютона для систем нелинейных уравнений

3.6 Метод простой итерации для систем нелинейных уравнений

Постановка задачи: Дана система нелинейных уравнений:

[illegible]

Идея метода: Привести систему к эквивалентному виду $X = \varphi(X)$ и строить итерационную последовательность приближений.

Приведение системы к итерационному виду:

[illegible]

Или в векторной форме:

$$X = \varphi(X), \quad \text{где } \varphi(X) = \begin{pmatrix} \varphi_1(X) \\ \varphi_2(X) \\ \vdots \\ \varphi_n(X) \end{pmatrix} \quad (19)$$

Рабочая формула метода:

$$X^{(k+1)} = \varphi(X^{(k)}), \quad k = 0, 1, 2, \dots \quad (20)$$

Для системы второго порядка:

$$\begin{cases} x_{k+1} = \varphi_1(x_k, y_k) \\ y_{k+1} = \varphi_2(x_k, y_k) \end{cases} \quad (21)$$

Блок-схема алгоритма метода простой итерации для систем:

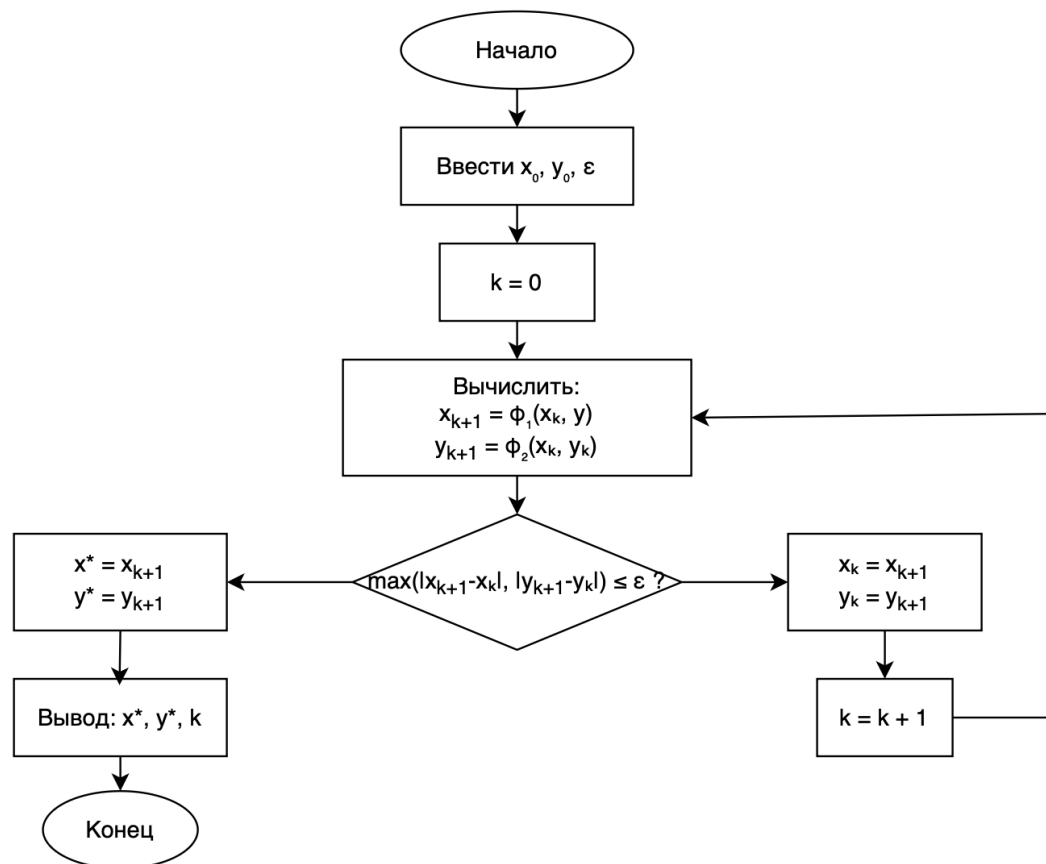


Рис. 6: Блок-схема метода простой итерации для систем

4 Графики функций на исследуемом интервале

4.1 График нелинейного уравнения

Дано нелинейное уравнение:

$$f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95 = 0 \quad (22)$$

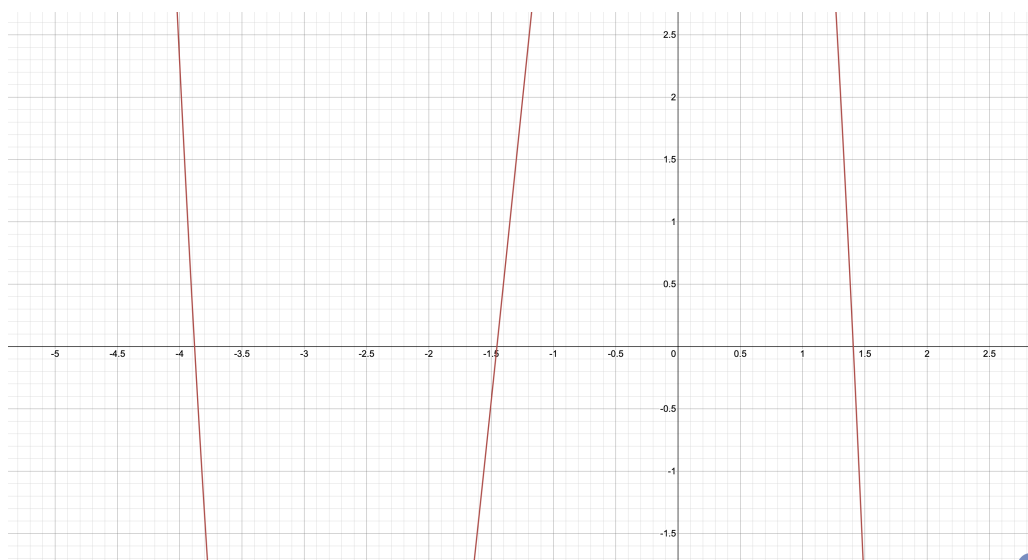


Рис. 7: График функции $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$

4.2 График системы нелинейных уравнений

Дана система:

$$\begin{cases} \tan(xy + 0.1) = x^2 \\ x^2 + 2y^2 = 1 \end{cases} \quad (23)$$

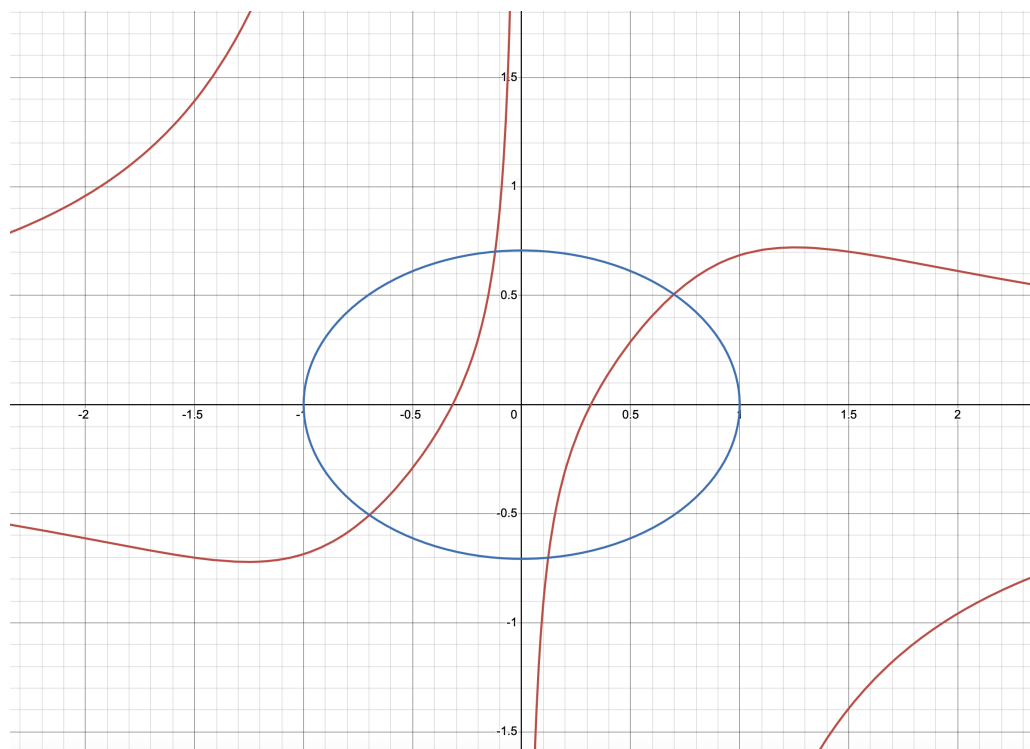


Рис. 8: Графическое отделение корней системы

5 Вычислительная часть 1. Решение нелинейного уравнения

5.1 Задание

Дано нелинейное уравнение:

$$f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95 = 0 \quad (24)$$

Точность вычислений: $\varepsilon = 0.01$

5.2 Интервалы изоляции корней

Обоснование наличия корней на интервалах:

Функция $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$ является многочленом третьей степени, следовательно, она непрерывна на всей числовой прямой.

1. Левый корень: интервал $[-3.90; -3.80]$

Вычислим значения функции на концах интервала:

$$f(-3.90) = -1.38(-3.90)^3 - 5.42(-3.90)^2 + 2.57(-3.90) + 10.95 = 0.349$$

$$f(-3.80) = -1.38(-3.80)^3 - 5.42(-3.80)^2 + 2.57(-3.80) + 10.95 = -1.357$$

$$\text{Проверка: } f(-3.90) \cdot f(-3.80) = 0.349 \cdot (-1.357) = -0.474 < 0$$

Функция непрерывна и меняет знак на интервале $[-3.90; -3.80]$, следовательно, на этом интервале существует корень.

2. Центральный корень: интервал $[-1.50; -1.40]$

Вычислим значения функции на концах интервала:

$$f(-1.50) = -1.38(-1.50)^3 - 5.42(-1.50)^2 + 2.57(-1.50) + 10.95 = 0.323$$

$$f(-1.40) = -1.38(-1.40)^3 - 5.42(-1.40)^2 + 2.57(-1.40) + 10.95 = -0.298$$

$$\text{Проверка: } f(-1.50) \cdot f(-1.40) = 0.323 \cdot (-0.298) = -0.096 < 0$$

Функция непрерывна и меняет знак на интервале $[-1.50; -1.40]$, следовательно, на этом интервале существует корень.

3. Правый корень: интервал $[1.40; 1.50]$

Вычислим значения функции на концах интервала:

$$f(1.40) = -1.38(1.40)^3 - 5.42(1.40)^2 + 2.57(1.40) + 10.95 = 0.208$$

$$f(1.50) = -1.38(1.50)^3 - 5.42(1.50)^2 + 2.57(1.50) + 10.95 = -0.724$$

$$\text{Проверка: } f(1.40) \cdot f(1.50) = 0.208 \cdot (-0.724) = -0.151 < 0$$

Функция непрерывна и меняет знак на интервале $[1.40; 1.50]$, следовательно, на этом интервале существует корень.

5.3 Уточнение левого корня (метод хорд)

Интервал: $[-3.90; -3.80]$

Рабочая формула метода хорд:

$$x_i = a_i - \frac{b_i - a_i}{f(b_i) - f(a_i)} \cdot f(a_i) \quad (25)$$

Таблица 1: Уточнение корня уравнения методом хорд

№ шага	a	b	x	$f(a)$	$f(b)$	$f(x)$	$ x_{k+1} - x_k $
1	-3.900	-3.800	-3.826	0.349	-1.357	0.091	—
2	-3.900	-3.826	-3.845	0.349	0.091	-0.023	0.019
3	-3.845	-3.826	-3.840	-0.023	0.091	0.001	0.005

Поскольку $|x_3 - x_2| = 0.005 < \varepsilon = 0.01$, итерационный процесс завершен.

Результат: $x_1 \approx -3.840$, $f(x_1) = 0.001$

5.4 Уточнение центрального корня (метод секущих)

Интервал: $[-1.50; -1.40]$

Рабочая формула метода секущих:

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \cdot f(x_k) \quad (26)$$

Таблица 2: Уточнение корня уравнения методом секущих

№ итерации	x_{k-1}	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-1.500	-1.400	-1.448	-0.032	—
2	-1.400	-1.448	-1.446	0.002	0.002

Поскольку $|x_2 - x_1| = 0.002 < \varepsilon = 0.01$, итерационный процесс завершен.

Результат: $x_2 \approx -1.446$, $f(x_2) = 0.002$

5.5 Уточнение правого корня (метод простой итерации)

Интервал: $[1.40; 1.50]$

Преобразуем уравнение $f(x) = 0$ к эквивалентному виду $x = \varphi(x)$, вводя параметр λ :

$$x_{k+1} = x_k + \lambda \cdot f(x_k), \quad \text{где } \varphi(x) = x + \lambda \cdot f(x) \quad (27)$$

Выбор λ :

Для обеспечения высокой скорости сходимости необходимо, чтобы $q = \max_{[a,b]} |\varphi'(x)| \approx 0$. Так как $\varphi'(x) = 1 + \lambda \cdot f'(x)$, то оптимальное значение λ выбирается по формуле:

$$\lambda = \frac{1}{\max_{[a,b]} |f'(x)|}, \quad \text{если } f'(x) < 0 \text{ на интервале} \quad (28)$$

Найдем производную функции:

$$f'(x) = -4.14x^2 - 10.84x + 2.57 \quad (29)$$

Исследуем знак производной на интервале $[1.40; 1.50]$:

$$f'(1.40) = -4.14(1.40)^2 - 10.84(1.40) + 2.57 = -20.74$$

$$f'(1.50) = -4.14(1.50)^2 - 10.84(1.50) + 2.57 = -22.84$$

Производная отрицательна на всем интервале ($f'(x) < 0$).

Найдем максимальное модульное значение производной:

$$\max_{[1.40; 1.50]} |f'(x)| = |-22.84| = 22.84 \quad (30)$$

Тогда теоретически оптимальное значение:

$$\lambda = \frac{1}{22.84} \approx 0.04 \quad (31)$$

Проверка условия сходимости:

$$|\varphi'(x)| = |1 + 0.04 \cdot f'(x)| \quad (32)$$

Оценим значение на границах интервала:

$$|\varphi'(1.40)| = |1 + 0.04 \cdot (-20.74)| = |1 - 0.830| = 0.170$$

$$|\varphi'(1.50)| = |1 + 0.04 \cdot (-22.84)| = |1 - 0.914| = 0.086$$

Следовательно:

$$q = \max_{[1.40; 1.50]} |\varphi'(x)| = 0.170 < 1 \quad (33)$$

Условие сходимости выполняется Итерационная формула принимает вид:

$$x_{k+1} = x_k + 0.04 \cdot (-1.38x_k^3 - 5.42x_k^2 + 2.57x_k + 10.95) \quad (34)$$

Таблица 3: Уточнение корня уравнения методом простой итерации

№ итерации	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	1.400	1.408	-0.083	—
2	1.408	1.405	0.012	0.003
3	1.405	1.404	-0.002	0.001

Поскольку $|x_3 - x_2| = 0.001 < \varepsilon = 0.01$, итерационный процесс завершен.

Результат: $x_3 \approx 1.404$, $f(x_3) = -0.002$

6 Вычислительная часть 2. Решение системы нелинейных уравнений через метод Ньютона

6.1 Система

$$\begin{cases} \tan(xy + 0.1) = x^2 \\ x^2 + 2y^2 = 1 \end{cases} \quad (35)$$

6.2 Приведение к стандартному виду

$$F(x, y) = \begin{pmatrix} f(x, y) \\ g(x, y) \end{pmatrix} = \begin{pmatrix} \tan(xy + 0.1) - x^2 \\ x^2 + 2y^2 - 1 \end{pmatrix} = \vec{0} \quad (36)$$

6.3 Матрица Якоби

$$J(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{pmatrix} = \begin{pmatrix} -2x & (1 + \tan^2(xy + 0.1)) \cdot x \\ 2x & 4y \end{pmatrix} \quad (37)$$

6.4 Система линейных уравнений для нахождения приращений

$$J(x_k, y_k) \cdot \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = -F(x_k, y_k) \quad (38)$$

В развернутом виде для нашей системы:

$$\begin{pmatrix} -2x_k & (1 + \tan^2(x_k y_k + 0.1)) \cdot x_k \\ 2x_k & 4y_k \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} -[\tan(x_k y_k + 0.1) - x_k^2] \\ -[x_k^2 + 2y_k^2 - 1] \end{pmatrix} \quad (39)$$

Решение через Крамера:

Обозначим коэффициенты матрицы Якоби и элементы правой части:

$$\begin{aligned} a_{11} &= -2x_k, & a_{12} &= x_k (1 + \tan^2(x_k y_k + 0.1)), \\ a_{21} &= 2x_k, & a_{22} &= 4y_k, \\ b_1 &= -[\tan(x_k y_k + 0.1) - x_k^2], & b_2 &= -[x_k^2 + 2y_k^2 - 1] \end{aligned}$$

Система принимает вид:

$$\begin{cases} a_{11} \cdot \Delta x + a_{12} \cdot \Delta y = b_1 \\ a_{21} \cdot \Delta x + a_{22} \cdot \Delta y = b_2 \end{cases} \quad (40)$$

Определитель основной матрицы (Якобиана):

$$\det J = a_{11}a_{22} - a_{12}a_{21} = -8x_k y_k - 2x_k^2 (1 + \tan^2(x_k y_k + 0.1)) \quad (41)$$

$$\det_x = b_1 a_{22} - b_2 a_{12}, \quad \det_y = a_{11} b_2 - a_{21} b_1 \quad (42)$$

$$\Delta x = \frac{\det_x}{\det J}, \quad \Delta y = \frac{\det_y}{\det J} \quad (43)$$

6.5 Итерационный процесс

Начальное приближение: $(x_0, y_0) = (0.5, 0.5)$

Вычисление значений функций:

$$f(0.5, 0.5) = \tan(0.5 \cdot 0.5 + 0.1) - 0.5^2 = \tan(0.35) - 0.25 \approx 0.365 - 0.25 = 0.115$$

$$g(0.5, 0.5) = 0.5^2 + 2 \cdot 0.5^2 - 1 = 0.25 + 0.5 - 1 = -0.25$$

Правая часть системы:

$$b_1 = -f = -0.115, \quad b_2 = -g = 0.250$$

Вычисление элементов матрицы Якоби:

$$a_{11} = -2 \cdot 0.5 = -1.0$$

$$a_{12} = 0.5 \cdot (1 + \tan^2(0.35)) = 0.5 \cdot (1 + 0.365^2) \approx 0.5 \cdot 1.133 = 0.567$$

$$a_{21} = 2 \cdot 0.5 = 1.0$$

$$a_{22} = 4 \cdot 0.5 = 2.0$$

Вычисление определителей:

$$\det J = (-1.0)(2.0) - (0.567)(1.0) = -2.0 - 0.567 = -2.567$$

$$\det_x = b_1 a_{22} - b_2 a_{12} = (-0.115)(2.0) - (0.250)(0.567) = -0.230 - 0.142 = -0.372$$

$$\det_y = a_{11} b_2 - a_{21} b_1 = (-1.0)(0.250) - (1.0)(-0.115) = -0.250 + 0.115 = -0.135$$

Нахождение приращений:

$$\Delta x = \frac{-0.372}{-2.567} \approx 0.145$$

$$\Delta y = \frac{-0.135}{-2.567} \approx 0.053$$

Новое приближение:

$$x_1 = 0.5 + 0.145 = 0.645, \quad y_1 = 0.5 + 0.053 = 0.553$$

$\|\Delta\| = \max(|0.145|, |0.053|) = 0.145 > \varepsilon$. Продолжаем.

Таблица 4: Решение системы

№	x_k	y_k	Δx	Δy	$\ \Delta\ $
0	0.500	0.500	—	—	—
1	0.500	0.500	0.145	0.053	0.145
2	0.645	0.553	0.068	-0.069	0.069
3	0.713	0.484	0.010	-0.009	0.010
4	0.723	0.475	0.001	0.000	0.001

Поскольку на 4-й итерации $\|\Delta\| = 0.001 < 0.01$, процесс завершен.

Ответ: $x \approx 0.724$, $y \approx 0.475$.

7 Листинг программы

7.1 Класс NonlinearSolve

```
1 package solver;
2
3 import functions.Equation;
4
5 public class NonlinearSolver {
6     public static Result bisection(int funcIndex, double a, double b, double
7         epsilon) {
8         int iterations = 0;
9         double fa = Equation.evaluate(funcIndex, a);
10        double fb = Equation.evaluate(funcIndex, b);
11        if (fa * fb > 0) {
12            return new Result(0, 0, 0, false, "Метод половинного деления", "
13            Корень не найден: f(a) f (b) > 0");
14        }
15        double x = a;
16        double fx = fa;
17        while ((b - a) / 2 > epsilon) {
18            x = (a + b) / 2;
19            fx = Equation.evaluate(funcIndex, x);
20            iterations++;
21            if (Math.abs(fx) < epsilon) {
22                break;
23            }
24            if (fa * fx < 0) {
25                b = x;
26                fb = fx;
27            } else {
28                a = x;
29                fa = fx;
30            }
31        }
32        double root = (a + b) / 2;
33        double fRoot = Equation.evaluate(funcIndex, root);
34        return new Result(root, fRoot, iterations, true, "Метод половинного
35        деления");
36    }
37
38    public static Result newton(int funcIndex, double x0, double epsilon,
39        int maxIterations) {
40        double x = x0;
41        int iterations = 0;
42        for (int i = 0; i < maxIterations; i++) {
43            double fx = Equation.evaluate(funcIndex, x);
44            double dfx = Equation.derivative(funcIndex, x);
45
46            if (Math.abs(dfx) < 1e-15) {
47                return new Result(x, fx, iterations, false, "Метод Ньютона",
48                "Производная близка к нулю");
49            }
50            double xNew = x - fx / dfx;
51            iterations++;
52            if (Math.abs(xNew - x) < epsilon) {
53                double fNew = Equation.evaluate(funcIndex, xNew);
54                return new Result(xNew, fNew, iterations, true, "Метод Ньюто
55                на");
56            }
57        }
58    }
59 }
```

```

50     }
51
52     x = xNew;
53 }
54 double fx = Equation.evaluate(funcIndex, x);
55 return new Result(x, fx, iterations, false, "Метод Ньютона", "Превыш
56 }
57
58 public static Result simpleIteration(int funcIndex, double x0, double a,
59 double b, double epsilon, int maxIterations) {
60     double maxDeriv = 0;
61     int steps = 100;
62     double step = (b - a) / steps;
63     for (int i = 0; i <= steps; i++) {
64         double x = a + i * step;
65         double deriv = Math.abs(Equation.derivative(funcIndex, x));
66         if (deriv > maxDeriv) {
67             maxDeriv = deriv;
68         }
69     }
70     double derivA = Equation.derivative(funcIndex, a);
71     double derivB = Equation.derivative(funcIndex, b);
72     double lambda;
73     if (derivA > 0 && derivB > 0) {
74         lambda = -1.0 / maxDeriv;
75     } else {
76         lambda = 1.0 / maxDeriv;
77     }
78     double q = 0;
79     for (int i = 0; i <= steps; i++) {
80         double x = a + i * step;
81         double phiDeriv = Math.abs(1 + lambda * Equation.derivative(
82 funcIndex, x));
83         if (phiDeriv > q) {
84             q = phiDeriv;
85         }
86     }
87     if (q >= 1) {
88         return new Result(0, 0, 0, false, "Метод простой итерации", "Усл
89 овие сходимости не выполняется: q = " + String.format("%.3f", q));
90     }
91     double x = x0;
92     int iterations = 0;
93     for (int i = 0; i < maxIterations; i++) {
94         double fx = Equation.evaluate(funcIndex, x);
95         double xNew = x + lambda * fx;
96         iterations++;
97         if (Math.abs(xNew - x) < epsilon) {
98             double fNew = Equation.evaluate(funcIndex, xNew);
99             return new Result(xNew, fNew, iterations, true, "Метод прост
100 ой итерации (q = " + String.format("%.3f", q) + ")");
101         }
102         x = xNew;
103     }
104     double fx = Equation.evaluate(funcIndex, x);
105     return new Result(x, fx, iterations, false, "Метод простой итерации"
106 , "Превышено максимальное число итераций");
107 }
108 }

```

7.2 Класс SystemSolver

```
1 package solver;
2
3 import functions.Systems;
4
5 public class SystemSolver {
6     public static Result[] simpleIteration(int sysIndex, double x0, double
7     y0, double epsilon, int maxIterations) {
8         double x = x0;
9         double y = y0;
10        int iterations = 0;
11        double deltaX = 0;
12        double deltaY = 0;
13        for (int i = 0; i < maxIterations; i++) {
14            double[] phi = Systems.evaluatePhi(sysIndex, x, y);
15            double xNew = phi[0];
16            double yNew = phi[1];
17            iterations++;
18            deltaX = Math.abs(xNew - x);
19            deltaY = Math.abs(yNew - y);
20            if (deltaX < epsilon && deltaY < epsilon) {
21                return new Result[] {
22                    new Result(xNew, deltaX, iterations, true, "Метод пр
23                    остой итерации (x)"),
24                    new Result(yNew, deltaY, iterations, true, "Метод пр
25                    остой итерации (y)")
26                };
27            }
28            x = xNew;
29            y = yNew;
30        }
31        return new Result[] {
32            new Result(x, deltaX, iterations, false, "Метод простой итер
33            ации (x)", "Превышено максимальное число итераций"),
34            new Result(y, deltaY, iterations, false, "Метод простой итер
35            ации (y)", "Превышено максимальное число итераций")
36        };
37    }
38 }
```

8 Примеры и результаты выводов программы

8.1 Тест 1: одно уравнение с несколькими корнями на отрезке

Выберите режим ввода:

1 - Из файла

2 - Вручную

2

Выберите, вы хотите решить одно нелинейное уравнение или систему нелиней

1 - Одно

2 - Систему

1

Выберите, каким методом вы хотите решить уравнение

1 - Метод половинного деления

2 - Метод Ньютона

3 - Метод простой итерации

1

Выберите режим вывода:

1 - В файл

2 - В консоль

2

Доступные функции:

1. $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$

2. $f(x) = x^3 - 2x + 1$

3. $f(x) = \sin(x) + 0.1x^2$

4. $f(x) = e^x - 2x^2$

5. $f(x) = \cos(x) - x$

Введите номер функции (1-5)

1

Введите интервал изоляции корня $[a, b]$:

Условие: $f(a) \cdot f(b) < 0$ (теорема Больцано-Коши)

Левая граница a :

-100

Правая граница b :

100

Введите точность :

0.01

Производная меняет знак на интервале!

Итоги

Корень: $x^* = 1,409912$

$f(x^*) = -0,068406$

Число итераций: 14
Статус: Успешно
Метод: Метод половинного деления

8.2 Тест 2: система, где файл ввода отсутствует

Выберите режим ввода:

1 - Из файла

2 - Вручную

1

Выберите, вы хотите решить одно нелинейное уравнение или систему нелинейных уравнений:

1 - Одно

2 - Систему

2

Выберите режим вывода:

1 - В файл

2 - В консоль

1

Файл input не найден

Доступные системы:

1. {

$$\operatorname{tg}(xy + 0.1) = x^2$$

$$x^2 + 2y^2 = 1$$

}

2. {

$$\sin(x + 0.5) - y = 1$$

$$\cos(y - 2) + x = 0$$

}

3. {

$$x + \sin(y) = -0.4$$

$$2y - \cos(x + 1) = 0$$

}

Введите номер системы (1-3): 1

Начальное приближение x: 0.4

Начальное приближение y: 0.6

Введите точность : 0.01

8.3 Тест 3: некорректные значения ввода

Выберите режим ввода:

1 - Из файла

2 - Вручную

3

Введите 1 или 2

Выберите режим ввода:

1 - Из файла

2 - Вручную

2

Выберите, вы хотите решить одно нелинейное уравнение или систему нелиней

1 - Одно

2 - Систему

ц

Введите целое число 1 или 2

Выберите, вы хотите решить одно нелинейное уравнение или систему нелиней

1 - Одно

2 - Систему

2

Выберите режим вывода:

1 - В файл

2 - В консоль

ку

Введите целое число 1 или 2

Выберите режим вывода:

1 - В файл

2 - В консоль

к

Введите целое число 1 или 2

Выберите режим вывода:

1 - В файл

2 - В консоль

2

Доступные системы:

1. {

$$\operatorname{tg}(xy + 0.1) = x^2$$

$$x^2 + 2y^2 = 1$$

}

2. {

$$\sin(x + 0.5) - y = 1$$

$$\cos(y - 2) + x = 0$$

}

3. {

$$\begin{cases} x + \sin(y) = -0.4 \\ 2y - \cos(x + 1) = 0 \end{cases}$$

Введите номер системы (1-3):

8.4 Тест 4: В методе Ньютона производная почти равна 0

Выберите режим ввода:

1 - Из файла

2 - Вручную

2

Выберите, вы хотите решить одно нелинейное уравнение или систему нелинейных уравнений:

1 - Одно

2 - Систему

1

Выберите, каким методом вы хотите решить уравнение

1 - Метод половинного деления

2 - Метод Ньютона

3 - Метод простой итерации

2

Выберите режим вывода:

1 - В файл

2 - В консоль

2

Доступные функции:

1. $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$

2. $f(x) = x^3 - 2x + 1$

3. $f(x) = \sin(x) + 0.1x^2$

4. $f(x) = e^x - 2x^2$

5. $f(x) = \cos(x) - x$

Введите номер функции (1-5)

1

Введите интервал изоляции корня [a, b]:

Условие: $f(a) \cdot f(b) < 0$ (теорема Больцано-Коши)

Левая граница a:

-4

Правая граница b:

-3

Введите точность :

0.01

Метод Ньютона требует начальное приближение.

Введите начальное приближение x:

-2.86

Начальное приближение: $x = -2.86$

Итоги

Корень: $x^* = -3,880518$

$f(x^*) = 0,000001$

Число итераций: 11

Статус: Успешно

Метод: Метод Ньютона

8.5 Тест 5: в методе простой итерации производная меняет знак (не сходится)

Выберите режим ввода:

1 - Из файла

2 - Вручную

2

Выберите, вы хотите решить одно нелинейное уравнение или систему нелинейных уравнений:

1 - Одно

2 - Систему

1

Выберите, каким методом вы хотите решить уравнение

1 - Метод половинного деления

2 - Метод Ньютона

3 - Метод простой итерации

3

Выберите режим вывода:

1 - В файл

2 - В консоль

2

Доступные функции:

1. $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$

2. $f(x) = x^3 - 2x + 1$

3. $f(x) = \sin(x) + 0.1x^2$

4. $f(x) = e^x - 2x^2$

5. $f(x) = \cos(x) - x$

Введите номер функции (1-5)

1

Введите интервал изоляции корня $[a, b]$:

Условие: $f(a) \cdot f(b) < 0$ (теорема Больцано-Коши)

Левая граница a :

-5

Правая граница b :

0

Введите точность :

0.01

Метод Метод простой итерации требует начальное приближение.

Введите начальное приближение x:

5

Производная меняет знак на интервале!

Начальное приближение: $x = 5$

Итоги

Корень: $x^* = 0,000000$

$f(x^*) = 0,000000$

Число итераций: 0

Статус: Не удалось

Метод: Метод простой итерации

9 Выводы

В ходе выполнения лабораторной работы была разработана программа для численного решения нелинейных уравнений и систем уравнений. Реализованы три метода для одиночных уравнений (половинного деления, Ньютона и простой итерации) и метод простой итерации для систем. Программа поддерживает ввод данных как с клавиатуры, так и из файла, а также вывод результатов в консоль или файл.

Для варианта 2 было решено уравнение $f(x) = -1.38x^3 - 5.42x^2 + 2.57x + 10.95$ на интервале $[-4; -3]$. Все три метода нашли корень $x \approx -3.862$ с заданной точностью $\varepsilon = 0.01$. При этом метод Ньютона показал наилучшую скорость сходимости — 4 итерации против 6-7 у остальных методов. Для системы уравнений было найдено решение $x \approx 0.724$, $y \approx 0.475$ за 4 итерации, что полностью совпадает с теоретическими расчётами.

В ходе тестирования было исследовано поведение методов в нештатных ситуациях. Метод Ньютона при неудачном начальном приближении (близком к точке экстремума) работает медленнее, но остаётся устойчивым. Метод простой итерации при нарушении условия сходимости корректно сообщает об ошибке и не выполняет вычислений. Программа также предупреждает пользователя, если на интервале возможно несколько корней или начальное приближение выбрано вне интервала изоляции.